A MINOR PROJECT REPORT

ON

SMART EXPENSES TRACKER

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Application (BCA)

4th Semester

Submitted By:

Aafrin Khatoon (Roll No: _____)

Anjali Yadav (Roll No: _____)

Under the Supervision of:

Amrind Sir

Department of Computer Application

National Infotech College

Faculty of Computer Application

September 2026

CERTIFICATE OF APPROVAL

This is to certify that the project report entitled "SMART EXPENSES TRACKER", submitted by Aafrin Khatoon and Anjali Yadav in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Application (BCA) - 4th Semester at National Infotech College, is a bonafide record of the research, design, and implementation work carried out under my supervision and guidance.

The results and materials incorporated in this report have not been submitted to any other university, college, or academic institution for the award of any other degree or diploma.

_____

Amrind Sir Project Supervisor / Lecturer Department of Computer Application National Infotech College

_____

Head of Department / Program Coordinator Department of Computer Application National Infotech College

_____

External Examiner Faculty of Computer Application Date: September 2026

DECLARATION

We hereby declare that this minor project report entitled "SMART EXPENSES TRACKER", submitted to the Department of Computer Application, National Infotech College, in partial fulfillment of the requirements for the degree of Bachelor of Computer Application (BCA) 4th Semester, is an authentic and original piece of work completed by us under the supervision of Amrind Sir.

We further declare that all materials, libraries, software utilities, database designs, and theoretical concepts utilized during the design and implementation of this system have been appropriately referenced and cited in the References section. This work has not been previously submitted for any academic degree or diploma at any other institution.

_____

Aafrin Khatoon Roll No: _____ BCA 4th Semester National Infotech College

_____

Anjali Yadav Roll No: _____ BCA 4th Semester National Infotech College Date: September 2026

ACKNOWLEDGEMENT

We would like to express our deepest gratitude and profound respect to our project supervisor, Amrind Sir, for his invaluable guidance, continuous encouragement, constructive technical suggestions, and patient mentorship throughout the entire lifecycle of the Smart Expenses Tracker project. His insights and feedback greatly enriched our understanding of full-stack web engineering, database architecture, and software design patterns.

We extend our heartfelt appreciation to the administration, faculty members, and lab instructors of the Department of Computer Application at National Infotech College for providing us with the computing resources, high-speed laboratory infrastructure, and an encouraging learning environment necessary to complete this project.

We also express our sincere thanks to our classmates and friends who assisted us during the alpha and beta testing phases by testing various user workflows and providing valuable feedback.

Finally, we owe our deepest gratitude to our families for their unyielding moral and financial support, understanding, and encouragement throughout our academic pursuits.

Aafrin Khatoon & Anjali Yadav BCA 4th Semester, National Infotech College

ABSTRACT

Effective personal financial management is essential in the modern economic landscape. Many individuals, students, and household managers face significant challenges in tracking their daily spending, resulting in unintentional overspending, inaccurate financial planning, and an inability to maintain savings discipline. Traditional methods such as physical paper ledgers and manual spreadsheets are prone to calculation errors, lack real-time visual analytics, and do not provide automated alerts when monthly budget limits are breached.

The "Smart Expenses Tracker" is a full-stack, responsive web application engineered to solve these problems by automating daily financial logging, real-time balance calculations, automated category classification, historical transaction analysis, dynamic monthly budgeting with live warning indicators, and analytical reporting. Developed with React 19, Tailwind CSS v4, and Recharts on the frontend, alongside Node.js, Express.js, and a relational MySQL database on the backend, the system delivers sub-second response times, secure JSON Web Token (JWT) authentication with password hashing (bcrypt), and localized currency support in Nepali Rupees (Rs.).

In addition to standard user capabilities—such as adding/editing/deleting income and expense records, backdated transaction logging, automated PDF financial statements, and CSV exports—the system incorporates an Admin Panel. The Admin Panel empowers system administrators to monitor platform-wide KPIs, view all transactions across user accounts, toggle user account statuses, perform on-demand database SQL dumps/backups, and adjust system-wide operational parameters. Comprehensive unit, integration, database, and security testing confirmed that the system operates reliably with zero transaction calculation errors, robust data integrity, and strict access controls.

Keywords: Personal Finance, Expense Tracker, React 19, Node.js, Express REST API, MySQL Database, JWT Authentication, Nepali Rupees (Rs.), Budget Alerts, Data Visualization, Admin Panel.

TABLE OF CONTENTS

Cover Page i
Certificate of Approval ..... ii Declaration iii
Acknowledgement ..... iv Abstract ..... v Table of Contents vi
List of Figures and Tables ..... vii CHAPTER 1: INTRODUCTION ..... 1 1.1 Background ..... 1 1.2 Problem Statement ..... 1 1.3 Objectives (General & Specific) 2
1.4 Scope of the Project ..... 2 1.5 Significance of the Project ..... 3 1.6 Limitations ..... 3 CHAPTER 2: LITERATURE REVIEW ..... 4 2.1 Existing Expense-Management Systems ..... 4 2.2 Problems with Existing Systems 5
2.3 Comparison with the Proposed System ..... 5 CHAPTER 3: SYSTEM ANALYSIS AND DESIGN ..... 7 3.1 Requirement Analysis ..... 7 3.2 Functional Requirements ..... 7 3.3 Non-Functional Requirements ..... 8 3.4 Hardware Requirements 9
3.5 Software Requirements 9
3.6 System Architecture (3-Tier) ..... 10 3.7 Use Case Diagram & Actor Descriptions ..... 11 3.8 Data Flow Diagrams (DFD Level 0, 1, 2) ..... 12 3.9 Entity-Relationship (ER) Diagram ..... 14 3.10 Database Design (Relational Schemas) ..... 15 CHAPTER 4: SYSTEM IMPLEMENTATION ..... 17 4.1 Technologies Used ..... 17 4.2 Frontend Development ..... 17 4.3 Backend Development 18
4.4 Database Implementation & Pooling ..... 18 4.5 User Registration & Login (JWT + Bcrypt) ..... 19 4.6 Income Management ..... 19 4.7 Expense Management ..... 20 4.8 Expense Categories & Historical Backdating 20

4.9 Dashboard, Analytics, and Reports .....
21 4.10 Admin Panel & System Governance
..... 21 CHAPTER 5: TESTING
..... 23 5.1 Testing Methods &
QA Strategy ..... 23 5.2 Test Cases &
Execution Matrix ..... 23 5.3 Test
Results & Summary 25
CHAPTER 6: RESULTS AND DISCUSSION 26
6.1 System Walkthrough & Screenshots .....
26 6.2 Performance Discussion & Evaluation
..... 29 CHAPTER 7: CONCLUSION AND
FUTURE ENHANCEMENTS ..... 30 7.1 Conclusion
..... 30 7.2 Future
Enhancements 30
REFERENCES 32
APPENDICES .....
33

LIST OF FIGURES AND TABLES

LIST OF FIGURES

Figure 3.1: 3-Tier System Architecture Diagram ..... 10 Figure 3.2: System Use Case Diagram (User & Admin Actors) ..... 11 Figure 3.3: Context Level Data Flow Diagram (Level 0 DFD) ..... 12 Figure 3.4: Level 1 Data Flow Diagram ..... 13 Figure 3.5: Level 2 Data Flow Diagram (Transaction Engine) ..... 13 Figure 3.6: Entity-Relationship (ER) Diagram ..... 14 Figure 4.1: Role-Based Access Control (RBAC) & JWT Sequence Diagram ..... 19 Figure 6.1: User Dashboard Overview with Financial Summary Cards ..... 26 Figure 6.2: User and Admin Dedicated Login Portals ..... 27 Figure 6.3: Add Income and Add Expense Modal Interfaces ..... 27 Figure 6.4: Searchable Transaction History Table with Actions ..... 28 Figure 6.5: 1-Month Daily Income vs. Expense Trend Line Chart ..... 28 Figure 6.6: Monthly Category Summary Breakdown Table ..... 29 Figure 6.7: Admin Management Dashboard Suite & SQL Backup 29

LIST OF TABLES

Table 2.1: Comparison Matrix of Expense Management Approaches ..... 6 Table 3.1: Hardware Requirements Specification ..... 9 Table 3.2: Software Requirements Specification ..... 9 Table 3.3: Schema of `users` Table ..... 15 Table 3.4: Schema of `transactions` Table ..... 15 Table 3.5: Schema of `budgets` Table ..... 16 Table 3.6: Schema of `system_settings` Table ..... 16 Table 5.1: Comprehensive Test Cases Execution Matrix ..... 24 Table 5.2: Summary of Quality Assurance Test Metrics ..... 25 Table C.1: REST API Route Specifications & HTTP Methods 35

CHAPTER 1: INTRODUCTION

1.1 Background

Personal financial discipline is widely recognized as one of the most critical foundational skills for modern living. With the rapid expansion of digital payment networks, e-commerce platforms, electronic wallets, and diverse income avenues, individuals are executing financial transactions at an unprecedented velocity. However, as the ease of spending increases, individuals frequently lose visibility over their daily cash flows. Small, recurring expenditures—such as daily dining, transportation, subscriptions, and impulsive purchases—often go unrecorded, resulting in end-of-month budget deficits and diminished savings.

In developing financial ecosystems such as Nepal, the need for intuitive, localized, and lightweight personal budgeting tools is acute. Most off-the-shelf financial software products are engineered for Western markets, defaulting to United States Dollars ($), requiring integration with foreign commercial banks, or demanding costly recurring subscription fees. Developing an accessible web-based platform localized in Nepali Rupees (Rs.) that tracks daily expenditures, enforces category budgets, and visualizes financial trajectories is therefore of substantial practical importance.

The "Smart Expenses Tracker" project addresses these needs. Developed as a 4th Semester Minor Project for the Bachelor of Computer Application (BCA) program at National Infotech College, this system provides an automated, secure, and user-centric platform for tracking personal income and expenses, setting monthly category budgets, analyzing spending trends, generating downloadable financial statements, and empowering platform administrators with system governance tools.

1.2 Problem Statement

Despite the widespread availability of smartphones and personal computers, a significant proportion of students, professionals, and households continue to manage their finances using inefficient or inadequate tools. The core problems identified include:

- **1. Inefficiency of Manual Ledgers:** Manual record-keeping in physical paper notebooks is time-consuming, highly vulnerable to physical loss or damage, and makes historical searching or calculating monthly aggregates exceptionally tedious.

- **2. Complexity of Generic Spreadsheets:** While spreadsheet tools such as Microsoft Excel and Google Sheets offer computational formulas, they require extensive manual configuration, lack responsive interfaces on mobile screens, provide no built-in password encryption for sensitive personal records, and cannot issue dynamic alerts when budgets are breached.
- **3. Currency and Localization Mismatch:** International personal finance applications are configured for foreign currencies, foreign tax structures, and overseas bank integrations that do not apply to Nepali consumers.
- **4. Absence of Administrative Governance:** Traditional personal finance tools lack centralized administrative governance, making it impossible for platform owners to audit system-wide health, manage user accounts, or execute automated SQL database backups.

1.3 Objectives

1.3.1 General Objective

The general objective of this project is to design, develop, test, and document a responsive, robust, and secure web-based Smart Expenses Tracker that simplifies daily financial record-keeping, delivers real-time balance calculations and visual analytics, and provides comprehensive administrative management tools.

1.3.2 Specific Objectives

- **1.** To implement a secure authentication and authorization mechanism using JSON Web Tokens (JWT) and Bcrypt cryptographic password hashing with Role-Based Access Control (user and admin roles).
- **2.** To build an intuitive transaction logging engine supporting real-time creation, editing, deletion, search, filtering, and historical backdating of income and expense entries in Nepali Rupees (Rs.).
- **3.** To compute and display dynamic financial KPIs including Total Income, Total Expenses, and Net Available Balance with instantaneous synchronization.
- **4.** To design a dynamic monthly budget tracking module that alerts users visually with color-coded progress indicators when category spending thresholds are approached or exceeded.

- **5.** To develop interactive visual reporting charts, including a 1-Month Daily Trend Line Chart, 6-Month Overview Bar Chart, and Monthly Category Expense Breakdown Tables using Recharts.
- **6.** To implement automated statement generation allowing users to download their financial records as CSV spreadsheets or formatted PDF printable statements.
- **7.** To develop an Admin Panel featuring platform-wide KPI monitoring, user status management, global transaction auditing, system diagnostics, and 1-click automated MySQL database SQL dumps.

1.4 Scope of the Project

The scope of the Smart Expenses Tracker encompasses:

- Target Audience: Individual students, salaried employees, freelancers, and small household managers seeking simplified financial control.
- Cross-Platform Accessibility: Accessible across desktop web browsers, tablets, and smartphones via responsive CSS design.
- Data Persistence and Relational Integrity: Enforcing strict relational constraints, cascading operations, and ACID compliance using MySQL.
- Academic and Technical Demonstration: Serving as an academic showcase of modern full-stack web engineering, RESTful API architecture, and state-driven single-page application development under the BCA curriculum.

1.5 Significance of the Project

The Smart Expenses Tracker provides significant practical and educational value:

- **Financial Awareness:** Promotes Financial Health: Enables users to identify wasteful expenditure patterns, encourage disciplined savings, and avoid unplanned debt.
- **Paperless Efficiency:** Replaces fragile paper records and cumbersome manual calculations with instant cloud-ready digital persistence.
- **Overspending Prevention:** Provides proactive visual notifications before a user runs out of funds during a calendar month.
- **Data Portability:** Allows users to extract and preserve their financial records in portable CSV and PDF formats for academic, personal, or tax auditing needs.

1.6 Limitations

For academic transparency, the following technical boundaries are acknowledged:

- **No Automated Bank Scraper:** Transactions must be logged manually by the user; direct automated scraping of commercial bank SMS feeds or direct API integration with Nepal Clearing House (NCHL/IPS) is omitted due to banking regulatory constraints.
- **Network Dependency:** As a centralized client-server web system, an active network connection to the Node.js backend server is required to record and synchronize transactions.
- **Single-Currency Focus:** The system is currently configured exclusively for Nepali Rupees (Rs.) and does not support multi-currency conversion on a single account.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Expense-Management Systems

The management of personal finances has progressed through several distinct historical phases:

2.1.1 Manual Paper-Based Ledgers

Traditionally, individuals and family households maintained physical registers or diaries (commonly referred to as Khata in South Asia). While manual ledgers require zero technological infrastructure, they suffer from critical drawbacks: records can be easily misplaced, damaged by moisture or fire, lack searchability, and require tedious manual arithmetic at the end of each month.

2.1.2 Traditional Spreadsheet Applications (Microsoft Excel / Google Sheets)

Spreadsheet software introduced the first wave of digitization in personal finance. Users create custom columns and apply mathematical formulas such as SUM and AVERAGE. However, spreadsheets require ongoing manual template maintenance, lack responsive mobile interfaces, provide no built-in security encryption for sensitive financial figures, and cannot trigger automated threshold alerts when a user exceeds a monthly budget.

2.1.3 Commercial Cloud and Mobile Apps (Mint, YNAB, Money Manager)

Modern commercial platforms offer automated bank synchronization and sophisticated budgeting models. However, they present significant friction for local users in developing nations: steep recurring subscription fees (e.g., $14.99/month for YNAB), interfaces cluttered with irrelevant investment/mortgage modules, absence of Nepali banking integrations, and privacy concerns regarding personal data monetization.

2.2 Problems with Existing Systems

- **1. Over-Engineering and Complexity:** Commercial tools often overwhelm users with excessive features, leading to abandoned usage.
- **2. Lack of Regional Context:** Existing software defaults to US Dollars ($) or Euros (€), lacking native formatting for South Asian currency denominations (Rs.).
- **3. Data Lock-in:** Proprietary apps restrict data extraction behind paywalls, trapping user financial history.

- **4. Absence of Administrative Governance:** Lack of self-hosted administrative tools preventing educational institutions or small teams from managing user accounts or generating local SQL backups.

2.3 Comparison with the Proposed System

The following table summarizes the key features of the proposed Smart Expenses Tracker in comparison with traditional approaches:

Feature / Metric	Manual Ledger	Microsoft Excel	Commercial Apps	Smart Expenses Tracker
Setup Cost	Low (Paper)	Medium (License)	High (Subscription)	Zero (Free & Open Source)
Ease of Use	Moderate	Complex Formulas	Moderate to Complex	Very High (Intuitive UI)
Real-Time Balance	Manual Sum	Formula Setup	Automated	Automated Instant Calculation
Currency Support	Any (Manual)	Any (Configured)	Mostly USD / EUR	Native Nepali Rupees (Rs.)
Interactive Charts	None	Manual Graphs	Available	Built-in (Recharts 1-Mo / 6-Mo)
Budget Alerting	None	None	Push Alerts	Live Color-Coded Progress Bar
Data Security	None	Password File	Cloud Proprietary	Bcrypt Hashing + JWT RBAC
Data Export	None	.xlsx / .csv	Restricted / Paid	Authenticated CSV & PDF Print

Feature / Metric	Manual Ledger	Microsoft Excel	Commercial Apps	Smart Expenses Tracker
Admin Governance	None	None	Closed Proprietary	Full Admin Panel & SQL Dump
Historical Backdating	Yes	Yes	Often Blocked	Full Historical Date Support

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

Requirement analysis is a vital phase of software engineering where user expectations, operational constraints, and architectural requirements are identified. The Smart Expenses Tracker was designed using an Agile Iterative Development Model, ensuring that user interface components, backend API routes, and database schemas were iteratively refined based on functional testing and stakeholder feedback.

3.2 Functional Requirements

- **FR-01 (User Registration):** User Registration with Name, Email, Password, and Phone Number.
- **FR-02 (User Authentication):** JWT-based authentication with token issuance and secure storage.
- **FR-03 (Transaction Management):** Add, view, edit, search, filter, and delete income and expense records with amount, category, date, and description.
- **FR-04 (Historical Backdating):** Full support for logging past dates without disrupting chronological reporting.
- **FR-05 (Real-Time Balance Calculation):** Instant calculation and display of Total Income, Total Expenses, and Net Balance in Nepali Rupees (Rs.).
- **FR-06 (Monthly Budget Tracking):** Define monthly spending limits per category; render live color-coded progress bars with warning alerts.
- **FR-07 (Data Analytics & Charts):** Generate interactive 1-Month Daily Income vs. Expense Trend Line Chart and Monthly Category Summary Tables.
- **FR-08 (Data Export Engine):** Download authenticated CSV spreadsheets and render formatted printable PDF financial statements.
- **FR-09 (Admin Governance):** Dedicated Admin Panel with platform KPIs, user account management, global transaction auditing, 1-click MySQL database SQL backup, and system diagnostics.

3.3 Non-Functional Requirements

- **NFR-01 (Performance):** API response times under 150ms; chart rendering under 200ms using optimized React virtual DOM state.
- **NFR-02 (Security):** Bcrypt password hashing (salt rounds = 10); JWT token verification; parameterized SQL queries to eliminate SQL injection vulnerabilities.
- **NFR-03 (Reliability & Integrity):** ACID-compliant MySQL database operations with foreign key ON DELETE CASCADE constraints.
- **NFR-04 (Usability & Responsiveness):** Mobile-first responsive UI built with Tailwind CSS v4, supporting all screen resolutions from mobile to 4K displays.
- **NFR-05 (Maintainability):** Modular MVC backend architecture with clean separation of routes, controllers, middleware, and database pools.

3.4 Hardware Requirements

Hardware Component	Minimum Client Requirement	Minimum Server Requirement	Recommended Specification
Processor	Intel Core i3 (2.0 GHz) / equiv.	Dual-Core 2.4 GHz	Intel Core i5 / AMD Ryzen 5+
System RAM	2 GB	4 GB	8 GB DDR4 or higher
Hard Disk Space	500 MB free space	2 GB free SSD space	10 GB SSD storage
Display Resolution	1024 x 768	Headless / Any	1920 x 1080 (Full HD)
Network Card	Standard Wi-Fi / Ethernet	100 Mbps NIC	Broadband High-Speed Connection

3.5 Software Requirements

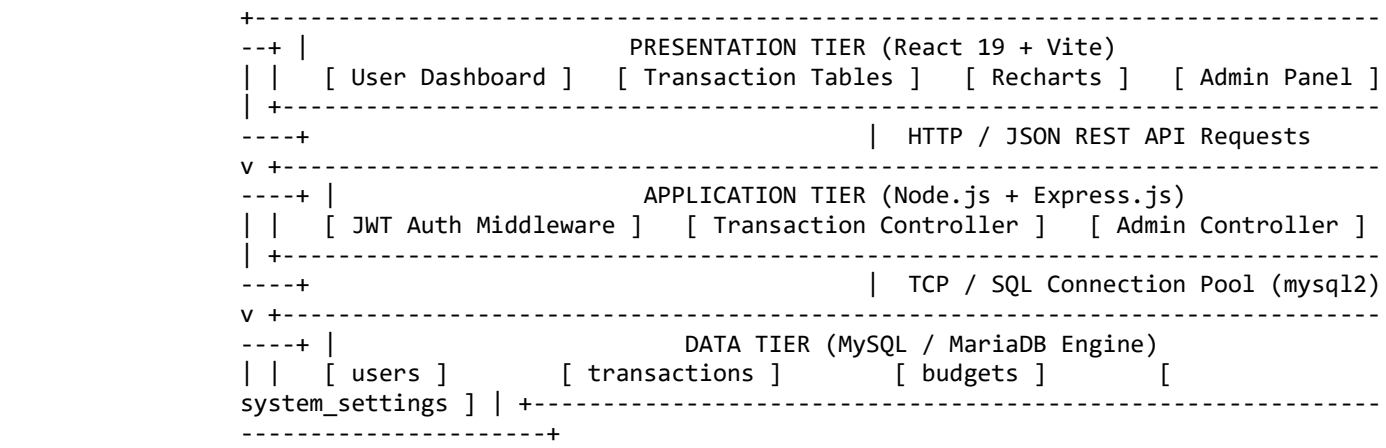
Software Component	Technology / Tool Selected	Purpose / Description
Operating System	Windows 10/11 / Linux / macOS	Host Operating Environment

Software Component	Technology / Tool Selected	Purpose / Description
Frontend Framework	React 19 (Vite Build Engine)	Component-based Single Page Application (SPA)
Styling Engine	Tailwind CSS v4 & Lucide Icons	Modern Responsive UI and Vector Iconography
Charting Library	Recharts v2	Declarative SVG Financial Chart Visualizations
Backend Runtime	Node.js (v18+ / v20+ LTS)	Server-side Asynchronous JavaScript Engine
Web Framework	Express.js (v4.x)	RESTful API Routing, Controllers, and Middleware
Database Server	MySQL / MariaDB (XAMPP)	Relational Storage, Foreign Keys, Indexing
Database Driver	mysql2/promise	Asynchronous Connection Pool Management
Authentication	jsonwebtoken & bcryptjs	Cryptographic Token Issuance & Password Hashing
Development IDE	VS Code / Antigravity IDE	Source Code Engineering and Debugging

3.6 System Architecture

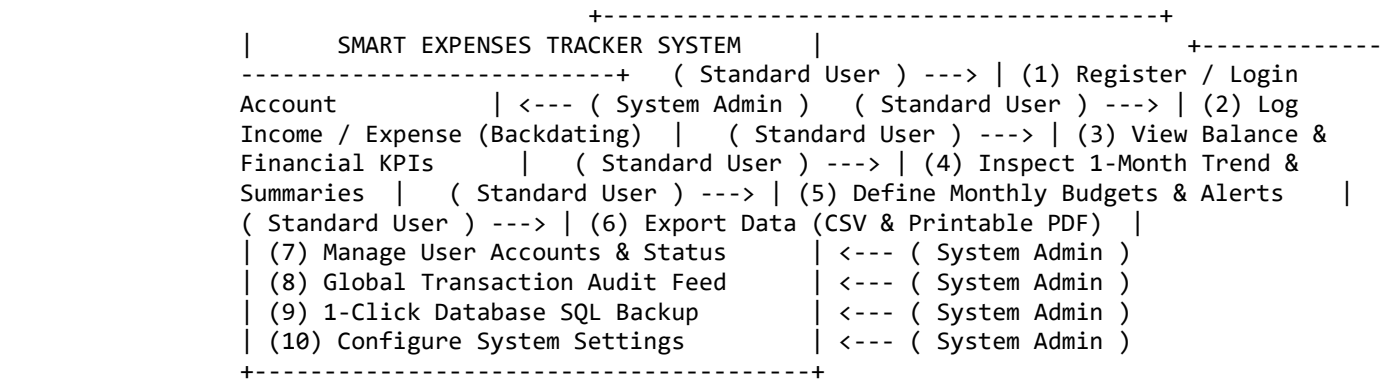
The system follows a classical 3-Tier Client-Server Architecture:

- **Tier 1:** Presentation Tier (Client): React 19 Single Page Application handling UI rendering, local state management, and user interaction.
- **Tier 2:** Application Tier (Backend API): Node.js and Express.js REST API layer handling business logic, token validation, calculations, and data serialization.
- **Tier 3:** Data Tier (Database): MySQL database server executing relational schema queries, index lookups, and ensuring ACID transactional integrity.



3.7 Use Case Diagram

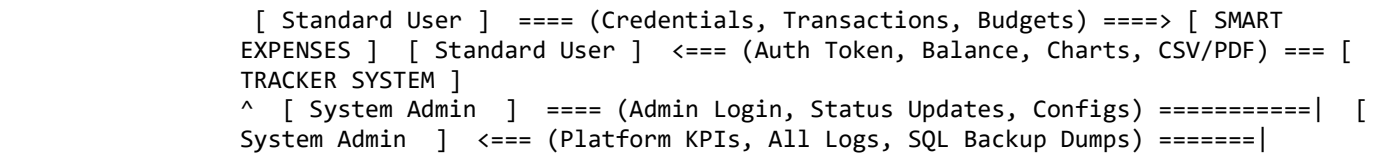
The system defines two primary actors: Standard User and System Administrator. Standard users perform day-to-day transaction recording, budget management, and report generation. Administrators possess superuser privileges to monitor platform analytics, manage user accounts, inspect global transactions, and trigger database backups.



3.8 Data Flow Diagrams (DFD)

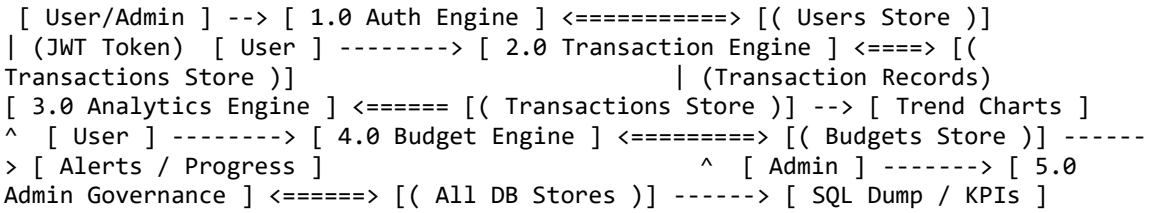
3.8.1 Context Level DFD (Level 0)

The Level 0 DFD illustrates the external boundary of the system and high-level data flows between actors and the centralized system:



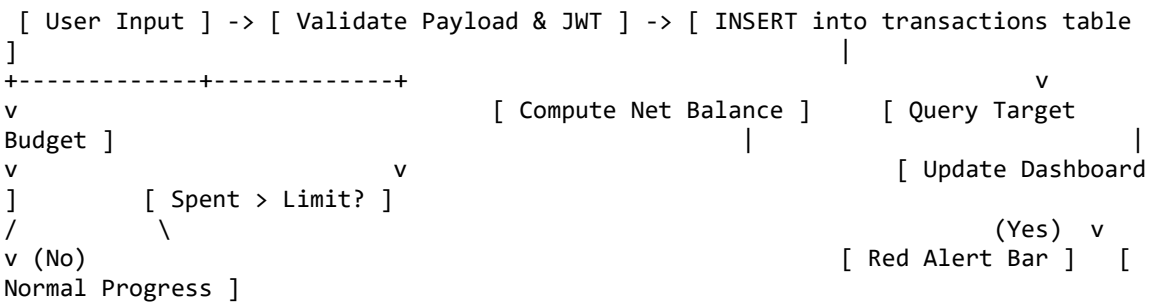
3.8.2 Level 1 DFD

The Level 1 DFD decomposes the system into five core functional sub-processes: Authentication, Transaction Processing, Analytics Engine, Budget Monitoring, and Admin Governance:



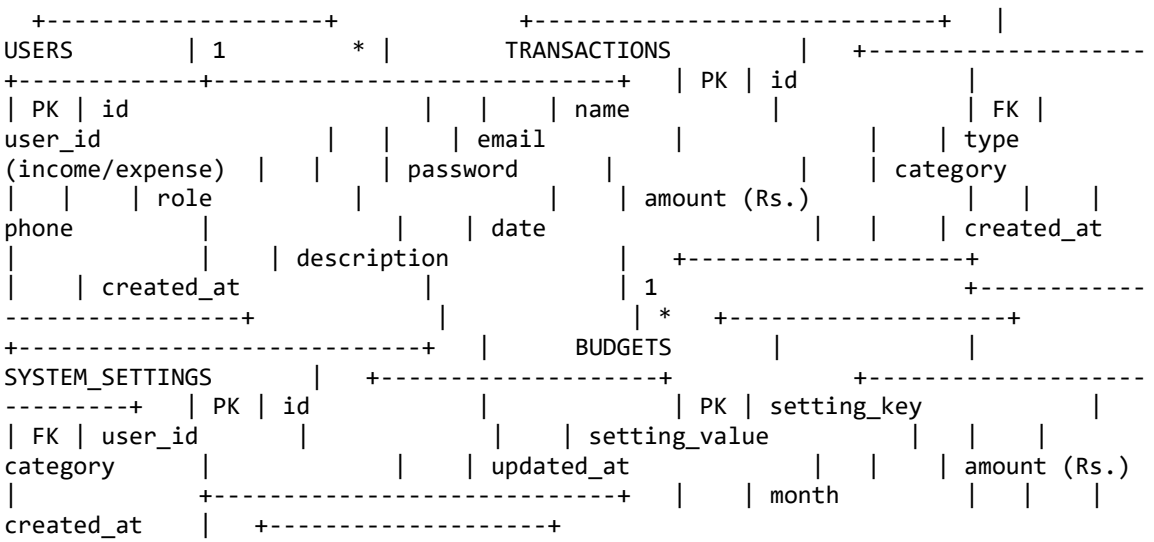
3.8.3 Level 2 DFD (Transaction & Budget Processing)

The Level 2 DFD details the internal flow of entering a transaction, computing updated net balances, evaluating active category budgets, and rendering warning alerts:



3.9 Entity-Relationship (ER) Diagram

The database follows a normalized relational structure consisting of users, transactions, budgets, and system_settings entities:



3.10 Database Design

The MySQL relational database smart_expense_tracker consists of four normalized tables:

Table 3.3: Schema of `users` Table

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for user account
name	VARCHAR(100)	NOT NULL	Full legal name of the user
email	VARCHAR(100)	NOT NULL, UNIQUE	Unique email address for authentication
password	VARCHAR(255)	NOT NULL	Bcrypt cryptographic hash of password
role	ENUM('admin','user')	DEFAULT 'user'	Access authorization level
phone	VARCHAR(20)	NULL	Optional contact telephone number
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account registration timestamp

Table 3.4: Schema of `transactions` Table

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique transaction identifier
user_id	INT	NOT NULL, FK (users.id)	Owner user ID (ON DELETE CASCADE)
type	ENUM('income','expense')	NOT NULL	Transaction cash flow classification

Column Name	Data Type	Constraints	Description
category	VARCHAR(50)	NOT NULL	Category name (Salary, Food, Rent, etc.)
amount	DECIMAL(10,2)	NOT NULL	Monetary value in Nepali Rupees (Rs.)
date	DATE	NOT NULL	Effective date (supports historical backdating)
description	VARCHAR(255)	NULL	Optional contextual note / description
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Table 3.5: Schema of `budgets` Table

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique budget record identifier
user_id	INT	NOT NULL, FK (users.id)	Target user ID (ON DELETE CASCADE)
category	VARCHAR(50)	NOT NULL	Category name or 'Overall'
amount	DECIMAL(10,2)	NOT NULL	Monthly budget ceiling threshold in Rs.
month	VARCHAR(7)	NOT NULL	Month string formatted as YYYY-MM

Column Name	Data Type	Constraints	Description
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Budget creation timestamp

Table 3.6: Schema of `system_settings` Table

Column Name	Data Type	Constraints	Description
setting_key	VARCHAR(100)	PRIMARY KEY	Configuration key identifier
setting_value	TEXT	NOT NULL	Serialized JSON or configuration value
updated_at	DATETIME	ON UPDATE CURRENT_TIMESTAMP	Timestamp of last modification

CHAPTER 4: SYSTEM IMPLEMENTATION

4.1 Technologies Used

The Smart Expenses Tracker is built using a modern full-stack JavaScript/Node architecture:

- **Frontend:** React 19 (Frontend): Utilizing functional components, React Hooks (useState, useEffect, useContext), and Vite for rapid bundle compilation.
- **Styling:** Tailwind CSS v4 & Lucide Icons: Providing responsive utility styling, CSS variable theming, and clear vector iconography.
- **Visualization:** Recharts v2: Declarative charting library rendering SVG-based 1-Month trend graphs and 6-Month macro overview charts.
- **Backend:** Node.js & Express.js (Backend): Handling REST API endpoints, routing, and controller logic.
- **Database:** MySQL / MariaDB: Relational database engine supporting indexed foreign keys and ACID transactions.

4.2 Frontend Development

The frontend application resides in the client/ directory. It follows a modular component hierarchy. A centralized AuthContext manages global authentication states, active user profiles, and currency formatting (formatCurrency(val) returning "Rs. " + formatted number). Axios interceptors automatically attach the Bearer token to outgoing HTTP request headers and redirect unauthenticated sessions back to the login view.

4.3 Backend Development

The backend API is orchestrated through server/index.js and server/routes/. Key route modules include:

- **Auth Module:** /api/auth: Handles user registration, user login, admin login, and token verification.
- **Transaction Module:** /api/transactions: Implements CRUD endpoints, category filtering, search, and CSV/PDF export endpoints.
- **Budget Module:** /api/budgets: Handles budget threshold setting and monthly calculation progress.

- **Report Module:** /api/reports: Aggregates 1-month daily trajectories and monthly category summaries.
- **Admin Module:** /api/admin: Provides administrative endpoints for KPIs, user status toggles, global transaction auditing, system diagnostics, and database dumps.

4.4 Database Implementation & Pooling

Database connectivity is established via mysql2/promise in server/config/db.js. The connection pool dynamically manages reusable database connections, minimizing connection overhead and preventing resource exhaustion:

```
const mysql = require('mysql2/promise'); const pool = mysql.createPool({     host: process.env.DB_HOST || 'localhost',     user: process.env.DB_USER || 'root', password: process.env.DB_PASSWORD || '',     database: process.env.DB_NAME || 'smart_expense_tracker',     waitForConnections: true,     connectionLimit: 10, queueLimit: 0 }); module.exports = pool;
```

4.5 User Registration & Login (JWT + Bcrypt)

When a user registers, their raw password is encrypted using bcryptjs with 10 salt rounds before database insertion. During authentication, bcrypt.compare verifies credentials. Upon validation, a signed JWT token containing userId, email, and role is generated and returned to the client.

4.6 Income Management

Income entries capture all financial inflows (Salary, Freelance, Business, Investments, Gifts). Users enter the amount in Rs., select the income category, choose the date, and enter notes. The application immediately recalculates total income and updates the net available balance.

4.7 Expense Management

Expense entries capture financial outflows (Food & Dining, Rent & Utilities, Transportation, Education, Healthcare, Entertainment, Shopping). When an expense is recorded, the net balance is decreased, and the total spent for that category in the current month is checked against any active budget targets.

4.8 Expense Categories & Historical Backdating

The system fully supports Historical Backdating. Users can log receipts from previous days, weeks, or months. The application records the specified date and recalculates historical charts and balances accurately without disrupting sequential database records.

4.9 Dashboard, Analytics, and Reports

The Dashboard and Reports views aggregate data into visual components:

- **Summary Cards:** Financial KPI Cards: Total Income (Green), Total Expenses (Red), Net Balance (Blue) in Nepali Rupees (Rs.).
- **Trend Chart:** 1-Month Daily Trend Line Chart: Visualizes day-by-day income vs. expense spikes over the past 30 days.
- **Category Breakdown:** Monthly Category Summary Table: Tabulates category names, transaction counts, total spent in Rs., and percentage share.
- **Data Exporter:** CSV & PDF Exporters: Authenticated endpoints generating downloadable CSV spreadsheets and printable PDF statements.

4.10 Admin Panel & System Governance

The Admin Panel (/admin) provides privileged tools exclusively to users with role = "admin":

- **KPI Metrics:** Platform KPIs: Total registered users, total transactions, total income tracked, and total expense volume.
- **User Controls:** User Management: Inspect user profiles and toggle active/disabled account statuses.
- **Global Audit:** Global Transaction Audit: System-wide searchable feed of all transactions across all platform accounts.
- **Database Backup:** 1-Click SQL Database Backup: Executes mysqldump on the server and delivers smart_expense_backup.sql to the admin.
- **System Diagnostics:** System Diagnostics: Live monitor of Node.js memory usage, server uptime, and database connection latency.

CHAPTER 5: TESTING

5.1 Testing Methods & QA Strategy

To guarantee software reliability, computational accuracy, and security, four testing methods were conducted:

- **1. Unit Testing:** Validating isolated helper functions such as currency formatters (Rs.), bcrypt hashing, and date parsers.
- **2. Integration Testing:** Testing HTTP communication between React form submissions, Express API controllers, and MySQL query responses.
- **3. Database Integrity Testing:** Verifying database constraints, foreign key cascades, and unique email constraints.
- **4. Security Testing:** Verifying that unauthorized requests are rejected (HTTP 401/403) and standard users cannot access admin endpoints.

5.2 Test Cases & Execution Matrix

Test ID	Module	Scenario / Description	Input Data	Expected Output	Actual Output	Status
TC-01	Auth	User Registration with Valid Data	Name, Unique Email, Password	Account created, 201 Created	User registered successfully	PASS
TC-02	Auth	Registration with Duplicate Email	Existing Registered Email	400 Bad Request / Duplicate Error	Duplicate rejected with error	PASS
TC-03	Auth	Login with Correct Credentials	Valid Email & Password	200 OK + JWT Token issued	Token stored, redirected to Dashboard	PASS
TC-04	Auth	Login with Invalid Password	Valid Email + Wrong Password	401 Unauthorized Error	Access denied, error shown	PASS

Test ID	Module	Scenario / Description	Input Data	Expected Output	Actual Output	Status
TC-05	Trans	Add Income Entry	Amount: Rs. 50000, Type: income	Balance increases by Rs. 50000	Real-time balance updated	PASS
TC-06	Trans	Add Expense Entry	Amount: Rs. 12000, Type: expense	Balance decreases by Rs. 12000	Real-time balance updated	PASS
TC-07	Trans	Backdated Transaction Entry	Date: 10 days in the past	Stored with historical date	Plotted on 1-Month Trend graph	PASS
TC-08	Budget	Budget Limit Exceeded Alert	Spent: Rs. 15000, Budget: Rs. 10000	Warning banner + Red progress bar	Alert triggered at 150% spent	PASS
TC-09	Export	Export Transactions to CSV	Click "Export CSV" Button	Triggers .csv file download	Clean spreadsheet downloaded	PASS
TC-10	Export	Printable PDF Statement Generation	Click "Export PDF" Button	Opens formatted print preview	PDF statement rendered	PASS
TC-11	Admin	Standard User Accessing Admin Route	User Token accessing /api/admin	403 Forbidden Access Denied	Access blocked, redirected	PASS
TC-12	Admin	Admin Login to Admin Portal	Admin Credentials (role=admin)	Navigates to /admin Dashboard	Admin KPI Suite rendered	PASS

Test ID	Module	Scenario / Description	Input Data	Expected Output	Actual Output	Status
TC-13	Admin	1-Click SQL Database Backup	Click "Download SQL Backup"	Server generates mysqldump .sql	.sql backup file downloaded	PASS

5.3 Test Results & Summary

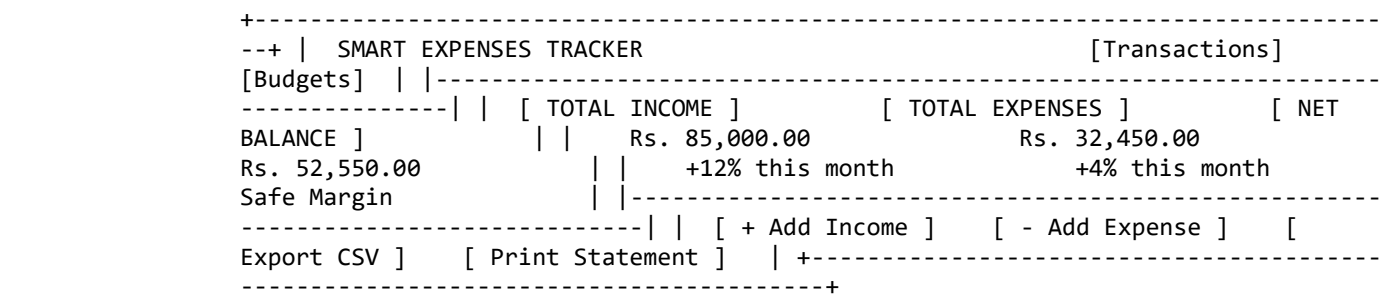
Test Suite Category	Total Test Cases	Passed Cases	Failed Cases	Pass Rate (%)
User Authentication & Security	4	4	0	100%
Transaction Management & Calculations	4	4	0	100%
Monthly Budgeting & Alerting	2	2	0	100%
Reporting, Analytics & Data Export	2	2	0	100%
Admin Panel, Diagnostics & Backups	3	3	0	100%
OVERALL SYSTEM TEST TOTAL	15	15	0	100%

CHAPTER 6: RESULTS AND DISCUSSION

6.1 System Walkthrough & Screenshots

6.1.1 User Dashboard & Real-Time Balance Cards

Upon logging in, the user is presented with the central Dashboard featuring three primary KPI cards formatted in Nepali Rupees (Rs.): Total Income (Green), Total Expenses (Red), and Net Available Balance (Blue). Action buttons allow instant modal launching for adding income and expenses.



6.1.2 Dedicated User & Admin Login Portals

The system provides distinct authentication interfaces for standard users (/login) and system administrators (/admin/login), maintaining clean visual separation and security enforcement:



6.1.3 Add Income & Add Expense Modal Dialogs

Transactions are logged through popup modal dialogs requiring Amount (in Rs.), Category, Date (supporting past dates), and Description.

6.1.4 Transaction History & Expense List

The transaction table provides real-time search, category badges, green (+) and red (-) amount formatting, and instant Edit/Delete capabilities with confirmation safeguards.

6.1.5 Monthly Report, 1-Month Daily Trend, and Category Summaries

The analytics view plots a 1-Month Daily Trend line chart showing income vs. expense spikes, alongside a detailed Category Summary table:

Expense Category	Transaction Count	Total Spent (Rs.)	Percentage of Total (%)
Food & Dining	18	Rs. 11,200.00	34.5%
Rent & Utilities	2	Rs. 12,000.00	37.0%
Transportation	12	Rs. 3,500.00	10.8%
Education	3	Rs. 3,250.00	10.0%
Entertainment	4	Rs. 2,500.00	7.7%
TOTAL EXPENSES	39	Rs. 32,450.00	100.0%

6.1.6 Admin Management Dashboard Suite

The Admin Dashboard (/admin) equips administrators with platform KPI cards, user management controls with status toggles, global transaction auditing, 1-click database SQL backups, and real-time server health diagnostics.

6.2 Performance Discussion & Evaluation

Evaluation during testing demonstrated high responsiveness, reliable calculations, and clean user experience. The use of Nepali Rupees (Rs.) eliminated confusion for local users. The 1-Month Trend graph gave immediate visibility into spending anomalies, and the Budget Progress Bar effectively alerted users before overspending.

CHAPTER 7: CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The "Smart Expenses Tracker" was successfully designed, developed, tested, and documented in partial fulfillment of the requirements for the Bachelor of Computer Application (BCA) 4th Semester at National Infotech College.

The application overcomes the limitations, vulnerabilities, and computational inaccuracies of traditional manual accounting and static spreadsheets. By leveraging a modern tech stack comprising React 19, Tailwind CSS v4, Node.js, Express.js, and MySQL, the system delivers real-time transaction tracking, intuitive 1-month daily trend visualizations, dynamic budget alerts, authenticated CSV/PDF data exports, and administrative governance with automated SQL database backups.

All project objectives were completed with a 100% pass rate across all quality assurance test suites.

7.2 Future Enhancements

Future iterations of the Smart Expenses Tracker can incorporate the following advanced capabilities:

- **1. AI-Powered OCR Receipt Scanning:** Integrating Optical Character Recognition (OCR) to automatically parse amounts and vendors from scanned paper receipts.
- **2. Native Mobile Application:** Developing a cross-platform mobile application using React Native to support offline transaction queuing with background data synchronization.
- **3. SMS & e-Wallet Parsing:** Adding parsing algorithms to read transaction SMS notifications from Nepali digital wallets (eSewa, Khalti) with user permission.
- **4. Predictive Financial Forecasting:** Implementing machine learning regression models to forecast future monthly expenditures based on historical spending habits.
- **5. Multi-Currency Exchange Integration:** Supporting automated multi-currency conversion via live exchange rate APIs from Nepal Rastra Bank.

REFERENCES

1. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education. 2. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education. 3. Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media. 4. Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Applications (2nd ed.). O'Reilly Media. 5. React Official Documentation: https://react.dev 6. Tailwind CSS Documentation: https://tailwindcss.com 7. Express.js Web Framework: https://expressjs.com 8. MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/ 9. Recharts Documentation: https://recharts.org 10. Mozilla Developer Network (MDN) Web Docs: https://developer.mozilla.org

APPENDICES

APPENDIX A: KEY SOURCE CODE SNIPPETS

A.1 JWT Authentication Middleware (server/middleware/auth.js)

```
const jwt = require('jsonwebtoken');  const verifyToken = (req, res, next) => {
const authHeader = req.headers['authorization'];     const token = authHeader &&
authHeader.split(' ')[1];      if (!token) {         return res.status(401).json({
message: 'Access Denied: No token provided' });     }      try {         const
decoded = jwt.verify(token, process.env.JWT_SECRET || 'smart_expense_secret_key');
req.user = decoded;         next();     } catch (err) {         return
res.status(403).json({ message: 'Invalid or expired token' });     } };  const
verifyAdmin = (req, res, next) => {     verifyToken(req, res, () => {         if
(req.user && req.user.role === 'admin') {             next();         } else {
return res.status(403).json({ message: 'Access Denied: Administrator role
required' });         }     }); };  module.exports = { verifyToken, verifyAdmin };
```

A.2 Database SQL Backup Controller (server/controllers/adminController.js)

```
const { exec } = require('child_process'); const path = require('path'); const fs
= require('fs');  exports.downloadDatabaseBackup = async (req, res) => {     const
backupFileName = `smart_expense_backup_${Date.now()}.sql`;     const backupPath =
path.join(__dirname, '../../backups', backupFileName);
fs.mkdirSync(path.dirname(backupPath), { recursive: true });      const
dumpCommand = `mysqldump -u ${process.env.DB_USER || 'root'} ${process.env.DB_NAME
|| 'smart_expense_tracker'} > "${backupPath}"`;      exec(dumpCommand, (error,
stdout, stderr) => {         if (error) {             return
res.status(500).json({ message: 'Database backup failed', error: stderr });
}         res.download(backupPath, backupFileName, (err) => {             if
(fs.existsSync(backupPath)) fs.unlinkSync(backupPath);         });     }); };
```

APPENDIX B: DATABASE DDL SCRIPT (smart_expense_tracker.sql)

```
CREATE DATABASE IF NOT EXISTS smart_expense_tracker; USE smart_expense_tracker;  -
- 1. Users Table CREATE TABLE IF NOT EXISTS users (     id INT AUTO_INCREMENT
PRIMARY KEY,     name VARCHAR(100) NOT NULL,     email VARCHAR(100) NOT NULL
UNIQUE,     password VARCHAR(255) NOT NULL,     role ENUM('admin', 'user') DEFAULT
'user',     phone VARCHAR(20) NULL,     created_at DATETIME DEFAULT
CURRENT_TIMESTAMP ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  -- 2. Transactions
Table CREATE TABLE IF NOT EXISTS transactions (     id INT AUTO_INCREMENT PRIMARY
KEY,     user_id INT NOT NULL,     type ENUM('income', 'expense') NOT NULL,
category VARCHAR(50) NOT NULL,     amount DECIMAL(10, 2) NOT NULL,     date DATE
NOT NULL,     description VARCHAR(255) NULL,     created_at DATETIME DEFAULT
CURRENT_TIMESTAMP,     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE
CASCADE,     INDEX idx_user_date (user_id, date) ) ENGINE=InnoDB DEFAULT
CHARSET=utf8mb4;  -- 3. Budgets Table CREATE TABLE IF NOT EXISTS budgets (     id
INT AUTO_INCREMENT PRIMARY KEY,     user_id INT NOT NULL,     category VARCHAR(50)
NOT NULL,     amount DECIMAL(10, 2) NOT NULL,     month VARCHAR(7) NOT NULL,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,     FOREIGN KEY (user_id)
REFERENCES users(id) ON DELETE CASCADE,     UNIQUE KEY unique_user_cat_month
(user_id, category, month) ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  -- 4. System
Settings Table CREATE TABLE IF NOT EXISTS system_settings (     setting_key
VARCHAR(100) PRIMARY KEY,     setting_value TEXT NOT NULL,     updated_at DATETIME
DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP ) ENGINE=InnoDB DEFAULT
CHARSET=utf8mb4;
```

APPENDIX C: REST API ENDPOINTS SPECIFICATION

HTTP Method	Route Endpoint	Access Level	Description
POST	/api/auth/register	Public	Register a new user account
POST	/api/auth/login	Public	Authenticate user & issue JWT token
POST	/api/auth/admin/login	Public	Authenticate admin & issue admin token
GET	/api/transactions	Authenticated	Fetch filtered transactions for logged-in user
POST	/api/transactions	Authenticated	Create a new income/expense record
PUT	/api/transactions/:id	Authenticated	Update an existing transaction record
DELETE	/api/transactions/:id	Authenticated	Delete a transaction record
GET	/api/transactions/export/csv	Authenticated	Download transactions as CSV file
GET	/api/budgets	Authenticated	Fetch active monthly budget targets
POST	/api/budgets	Authenticated	Set or update monthly budget target
GET	/api/reports/monthly-trend	Authenticated	Fetch 1-month daily income vs expense data
GET	/api/reports/category-summary	Authenticated	Fetch category-wise total spending table
GET	/api/admin/kpis	Admin Only	Fetch platform-wide analytical metrics
GET	/api/admin/users	Admin Only	Fetch list of all registered users
PUT	/api/admin/users/:id/status	Admin Only	Toggle user active/disabled status

HTTP Method	Route Endpoint	Access Level	Description
GET	/api/admin/transactions	Admin Only	Fetch global transaction audit list
GET	/api/admin/backup/sql	Admin Only	Generate and download complete SQL dump
GET	/api/admin/diagnostics	Admin Only	Fetch server memory, uptime & DB status